Article

ubms ai 1: 반야수 - 2차원 수체계와 GPU 없는 LLM 연산

한혁진 (Hyukjin Han)

독립 연구자, 경기도 화성, 대한민국; bokkamsun@gmail.com

Abstract: 0이 아닌 모든 수는 $e^{\ln x}$ 형태로 표기할 수 있다. 여기서 출발해서 곱셈 없이, GPU 없이 LLM을 실행하는 정수 전용 연산 체계를 만든다. 본 논문에서 제안하는 것은 세 가지다. 첫째, 반야수. 수의 본질은 2차원이다. 크기와 각도를 단일 단위로 보유하는 수를 반야수라 부른다. 가중치도 활성값도 모두 반야수다. 연산은 유니터리 회전만으로 처리하므로 변환도 손실도 없다. 둘째, 식 자체를 값으로 보유한다. 반야수의 합은 실수로 축약하지 않고 e 지수 항들의 정수 계수 배열로 보존한다. e가 초월수이므로 같은지 비교는 계수 배열만 보면 정확하다. 셋째, 미분은 회전이다. e 지수 형태는 미분해도 형태가 변하지 않고, 미분 한 번은 90도 회전이다. 분수 미분은 임의 각도 회전이고, 반야프레임의 분수 미분 회전과 같은 구조다. 결과적으로 곱셈기도 부동소수점 장치도 없이 덧셈, 시프트, 비교만으로 LLM 전 과정을 실행한다.

Keywords: banya number; two-dimensional number; logarithmic number system; unitary rotation; fractional derivative; integer-only inference; multiplication-free LLM; SIMD; bounded activation; transcendence of e

---

1. 들어가며

이 체계는 아래 순서로 도출되었다. 각 단계는 앞 단계에서 봉착한 문제를 해결하면서 도출되었다.

1. 모든 수는 $e^{\ln x}$ 형태로 표기할 수 있다.
2. 큰 수 계산을 전부 지수 더하기로 하면 곱셈이 필요 없다. 1614년 네이피어의 로그 [1]와 같은 위치다.
3. 보정값을 표로 만들어 두면 비용은 조회(매핑) 속도뿐이다.
4. 로그값을 정수로 두면 부동소수점이 필요 없다.
5. 목표는 GPU 없는 곳에서 실행되는 LLM이다.
6. 덧셈한 결과를 표로 로그에 복원하지 않고 그대로 유지한다.
7. 값을 실수나 정수로 변환하지 않고 식 그 자체를 새 자료형으로 간주한다.
8. top-k는 값이 아니라 순서만 필요하다.
9. e에 회전 i를 곱하면 그것이 미분이다. 그래서 소프트맥스도 쉽다.
10. 수의 본질은 2차원이다. 가중치도 1차원 실수가 아니다. 이 수를 반야수라 한다.
11. 시그모이드, tanh 같은 유계 함수도 변환 없이 처리한다.
12. 반야수는 32비트 공용체 하나에 담는다.

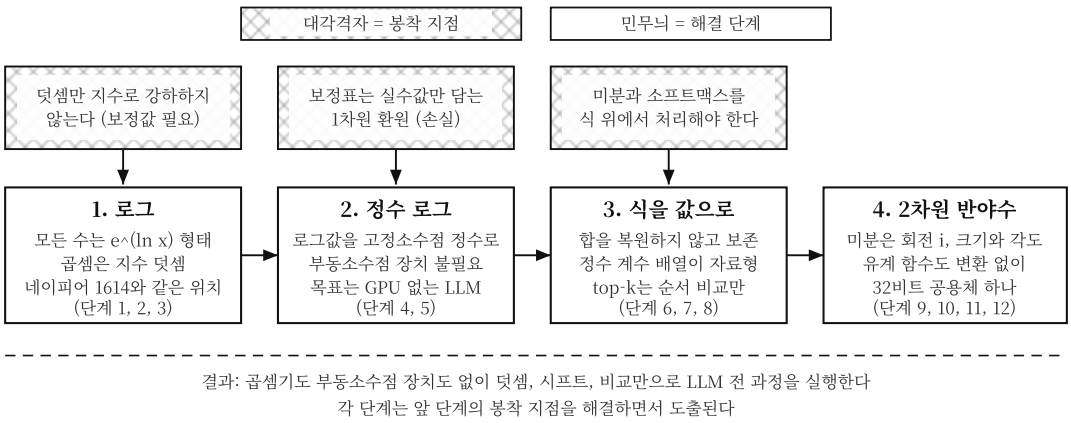


Figure 1. 체계의 도출 사슬 - 로그에서 출발해 정수 로그, 식을 값으로 보존하는 계수 배열, 2차원 반야수의 네 단계로 진행한다. 각 단계는 앞 단계의 봉착 지점(대각격자)을 해결하면서 도출된다.

2. 수학 기초

양수는 지수와 로그가 서로 역함수이므로 바로 성립한다.

$$x = e^{\ln x} \quad (x > 0) \tag{1}$$

음수는 오일러 공식을 사용한다. $e^{i\pi} = -1$이므로 양수를 180도 회전하면 음수다. 예를 들어 $-5 = e^{\ln 5 + i\pi}$이다.

$$e^{i\theta} = \cos\theta + i\sin\theta, \qquad -x = e^{\ln x + i\pi} \tag{2}$$

지수의 실수부는 크기를 정하고 허수부는 방향을 정한다. 2π를 더 회전해도 같은 위치이므로 표현은 무한히 많다. 구현에서는 각도를 한 바퀴 안에서만 다룬다.

0은 예외다. e를 몇 제곱해도 0은 나오지 않는다. 구현에서는 크기 필드(구조체 안에서 값 하나를 담는 위치)의 최솟값을 0으로 약속한다.

지수로 옮기면 연산이 한 단계씩 강하한다.

Table 1. 지수로 옮겼을 때 연산의 강하.

원래 연산	지수에서
a 곱하기 b	$\ln a + \ln b$
a 나누기 b	$\ln a - \ln b$
a의 n제곱	n 곱하기 $\ln a$
a의 제곱근	$\ln a$ 나누기 2
$a + b$	큰 쪽 + 보정값

덧셈만 강하하지 않는다. 본 논문의 나머지는 대부분 이 덧셈을 어떻게 처리하느냐에 관한 서술이다.

3. 반야수: 2차원 수와 회전

반야수는 크기와 각도를 단일 단위로 보유하는 2차원 수다. 가중치, 활성값, 중간값 전부 반야수다.

$$z = e^{a+i\theta}, \qquad a = \ln|z|, \quad \theta = \text{각도} \tag{3}$$

3.1. 1차원 환원은 손실이 있다

실수로 변환하면 각도가 폐기된다. 보정표(LUT)는 실수값만 저장하므로 결국 1차원 환원이고 손실이 생긴다. 실수 미분도 1차원 선형변환이라 상수항을 없앤다. 그래서 적분으로 복원할 때 상수 C가 소실된다.

3.2. 회전은 손실이 없다

$e^{i\varphi}$를 곱하는 회전은 유니터리다. 크기를 그대로 두고, 반대로 회전하면 원래대로 복원된다. 반야수의 연산은 이 회전으로 끝난다.

Table 2. 반야수의 연산과 역연산.

연산	반야수에서	역연산(원래 값으로 복원하는 연산)
곱셈	크기끼리 더하고 각도끼리 더한다	뺄셈
미분	90도 회전한다	−90도
적분	−90도 회전한다	90도
α차 분수 미분	α 곱하기 90도 회전한다	반대로

미분하고 적분하면 원래 값으로 그대로 복원된다.

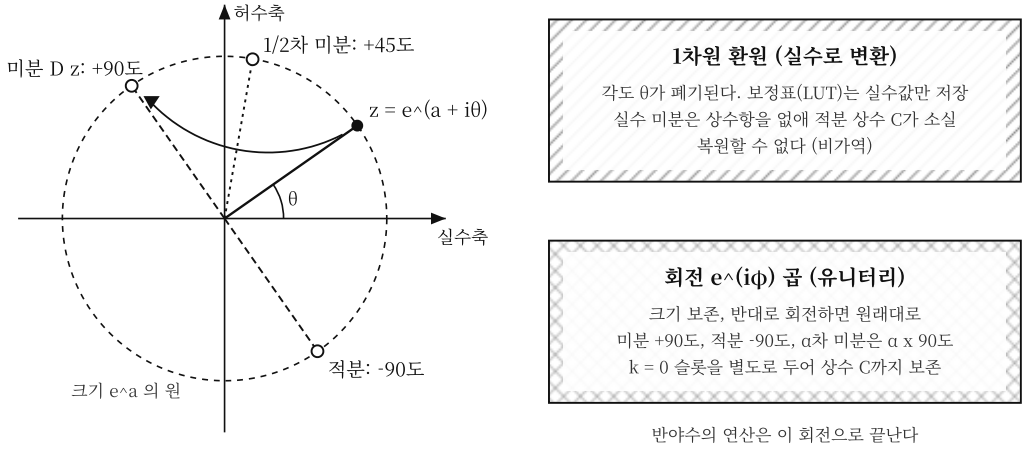


Figure 2. 반야수의 2차원 구조와 회전 연산 - 복소평면 위에서 크기 e^a와 각도 θ가 한 점을 정하고, 미분은 $+90$도, 적분은 -90도, α차 분수 미분은 $\alpha \times 90$도 회전이다. 실수축으로의 1차원 환원(빗금)은 각도를 소실하고, 회전(대각격자)은 크기를 보존한다.

3.3. 상수 C도 소실되지 않는다

미분은 e^{ikx} 항에 ik를 곱한다. k가 0이 아닌 항은 회전(복소평면에서 크기는 그대로 두고 각도만 변경하는 연산)으로 생존한다. $k = 0$인 항, 곧 상수항만 0이 곱해져서 사라진다. 이것이 적분 상수 C의 정체다. 반야수는 값을 계수 배열로 보유하므로 $k = 0$ 슬롯(계수 배열에서 지수 하나에 대응하는 위치)을 별도로 두면 C까지 보존한다.

3.4. 가중치도 반야수다

가중치는 1차원 실수가 아니다. 크기와 각도를 가진 반야수다. 가중치를 곱하는 것은 크기 조절과 회전이다. 현재 사용하는 실수 가중치는 각도가 0 아니면 π로 고정된 특수한 경우일 뿐이다.

3.5. 분수 미분과 반야프레임

미분 한 번이 i이고 i는 90도다. α번 미분은 i의 α제곱이다. $1/2$차 미분은 45도다.

$$D^\alpha e^{ix} = i^\alpha e^{ix} = e^{i\pi\alpha/2}\, e^{ix} \tag{4}$$

정수 미분과 분수 미분이 회전 하나로 합쳐진다. 반야프레임의 분수 미분 회전과 같은 구조다 [11].

4. 반야수 자료형

반야수 하나는 32비트 공용체 하나에 담는다. 같은 32비트를 raw(크기와 각도로 나누지 않은 32비트 원본 그대로)로도 보고 크기와 각도 두 필드로도 본다. 2차원 수를 단일 단위로 보유하는 반야수와 구조가 같다.

4.1. 배치

Table 3. 32비트 공용체의 비트 배치.

비트	내용	형식
31~16	각도	부호 없는 16비트, 65536이 2π
15~0	크기 $\ln \lvert z \rvert$	부호 있는 16비트, Q8.8 고정소수점

x86과 ARM은 리틀엔디언이라 먼저 선언한 크기가 하위, 각도가 상위에 놓인다. 각도가 상위에 있으니 raw로 더했을 때 각도에서 오버플로(덧셈 결과가 비트 수를 넘어 윗부분이 잘려 나가는 것)된 비트는 32비트

밖으로 폐기된다. 2π 순환(wrap-around, 각도가 한 바퀴 360도를 넘으면 0부터 다시 세는 것)을 하드웨어가 추가 비용 없이 수행한다. 구조체 전체가 4바이트라 함수 인자로 넘길 때 레지스터 하나에 적재된다.

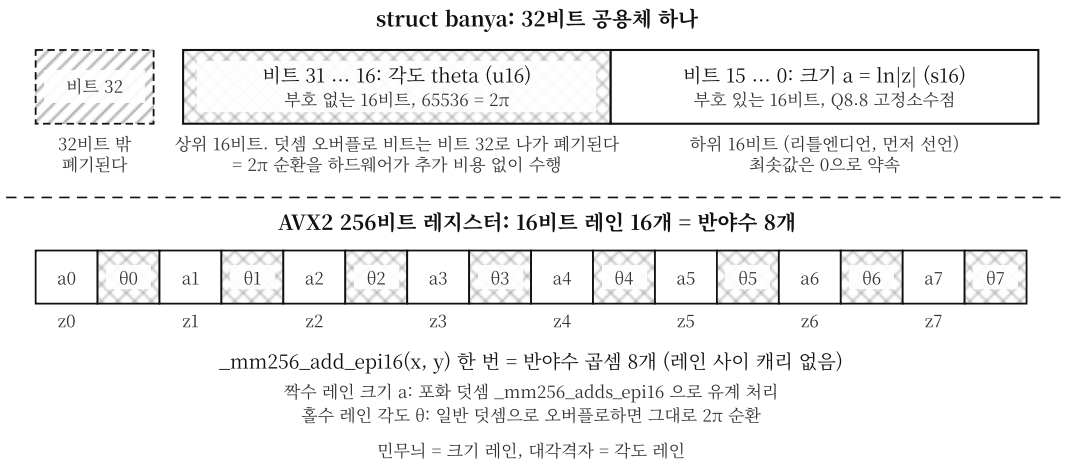


Figure 3. 32비트 공용체의 비트 배치와 SIMD 적재 - 상위 16비트 각도의 오버플로 비트는 32비트 밖으로 폐기되어 2π 순환이 되고, AVX2 256비트 레지스터의 16비트 레인 16개에 반야수 8개가 크기와 각도 레인으로 번갈아 적재되어 명령 하나가 곱셈 8개를 수행한다.

4.2. 정의

```
typedef unsigned int   u32;
typedef short          s16;
typedef unsigned short u16;

struct banya {
    union {
        u32 raw;            // raw 32비트, 내부에서만 사용한다
        struct {
            s16 a;          // 크기 ln|z|, Q8.8
            u16 theta;      // 각도, 65536 = 2pi
        } p;
    };

    // 곱셈: 크기끼리, 각도끼리 더한다
    banya operator*(const banya& y) const {
        banya r;
        r.p.a     = p.a + y.p.a;
        r.p.theta = p.theta + y.p.theta;
        return r;
    }

    // 나눗셈: 크기끼리, 각도끼리 뺀다
    banya operator/(const banya& y) const {
        banya r;
        r.p.a     = p.a - y.p.a;
        r.p.theta = p.theta - y.p.theta;
        return r;
    }

    // 미분 90도, 적분 -90도
    banya deriv() const { banya r = *this; r.p.theta += 16384; return r; }
    banya integ() const { banya r = *this; r.p.theta -= 16384; return r; }
```

```
    // 분수 미분: alpha_q 는 alpha 를 1/16384 단위 정수로
    banya frac_deriv(int alpha_q) const {
        banya r = *this; r.p.theta += (u16)alpha_q; return r;
    }

    // 같은지 비교: raw 비교 한 번
    bool operator==(const banya& y) const { return raw == y.raw; }

    // 정수 변환 연산자는 일부러 만들지 않는다
};
```

32비트를 정수로 읽으면 각도와 크기가 섞인 의미 없는 수가 나온다. 반야수는 정수의 변형이 아니라 독립된 수체계이므로 정수 변환을 차단한다. 그러면 반야수를 정수처럼 사용하는 코드는 컴파일에서 차단된다. raw는 대량 연산에서 레지스터에 적재할 때만 사용한다.

4.3. SIMD 일괄 처리

x86 SIMD의 16비트 레인(SIMD 레지스터를 같은 크기로 나눈 각 구역) 덧셈은 레인끼리 캐리(덧셈에서 윗자리로 올라가는 1)가 넘어가지 않는다. 반야수의 크기 필드와 각도 필드가 그대로 16비트 레인에 부합한다. 명령 하나가 반야수 곱셈 여러 개를 한 번에 수행한다.

Table 4. SIMD 명령 하나당 반야수 곱셈 개수.

명령	레지스터	명령 하나당 반야수 곱셈
MMX paddw (1997)	64비트	2개
SSE2 _mm_add_epi16	128비트	4개
AVX2 _mm256_add_epi16	256비트	8개
AVX-512 _mm512_add_epi16	512비트	16개

```
#include <immintrin.h>

// 반야수 8개와 8개를 명령 하나로 곱한다
// 짝수 레인 크기, 홀수 레인 각도
__m256i banya_mul8(__m256i x, __m256i y)
{
    return _mm256_add_epi16(x, y);
}
```

각도 레인은 일반 덧셈이라 오버플로되면 그대로 순환된다. 크기 레인은 포화 덧셈을 사용하면 오버플로될 때 최댓값에서 멈춘다(`_mm256_adds_epi16`). 8장의 유계 처리가 명령 하나로 들어간다. 두 명령을 레인 마스크로 섞어서 크기는 포화, 각도는 순환으로 구분한다.

4.4. 기존 자료형과 비교

Table 5. 기존 복소수 자료형과 banya의 비교.

자료형	저장	곱셈 한 번
C99 _Complex, C++ std::complex	실수부와 허수부	실수 곱셈 4번, 덧셈 2번
banya	크기 로그와 각도	16비트 정수 덧셈 2번
banya + AVX2	8개 팩(레지스터 하나에 8개를 같이 실은 것)	명령 하나로 8번

크기와 각도로 보유하는 기성 자료형은 C에도 C++에도 없다. banya가 그 위치다.

5. 계수 표현: 식을 값으로 보유한다

반야수 두 개를 더하면 크기 하나, 각도 하나로 합쳐지지 않는다. 여기서 실수로 변환하면 1차원으로 환원되어 손실이 생긴다. 그래서 합은 합친 값이 아니라 식 그대로 보유한다.

5.1. 정의

지수 스텝(지수를 나누는 최소 간격)을 $1/256$로 잡고 $q = e^{1/256}$이라 하면 값은 이렇게 쓴다.

$$v = \sum_{k} c_k \, q^{k}, \qquad q = e^{1/256}, \quad c_k\text{는 정수} \tag{5}$$

값은 지수 슬롯마다 붙은 정수 계수 배열이다.

```
struct banya_sum {
    int  lo;   // 가장 낮은 지수 슬롯 번호
    int  n;    // 슬롯 개수
    int* c;    // 슬롯별 정수 계수, 값 = sum c[i] * e^((lo+i)/256)
};
```

Table 6. 계수 배열 위의 연산과 오차.

연산	처리	오차
덧셈, 뺄셈	같은 슬롯 계수끼리 더한다	없음
곱셈	계수 배열끼리 다항식 곱, 지수는 더해진다	없음
한 항으로 나눗셈	슬롯을 옮긴다	없음
같은지 비교	계수 배열 비교	없음
크기 비교	큰 슬롯부터 검사한다	결과는 정확
정규화	작은 슬롯을 절단한다	정한 만큼만

5.2. 같은지 비교는 정확하다

$q = e^{1/256}$은 초월수다. 0이 아닌 대수적 수 a에 대해 e^{a}는 초월수라는 린데만-바이어슈트라스 정리 때문이다. 따라서 0이 아닌 정수 계수 다항식은 q에서 0이 될 수 없다. 계수 배열이 다르면 값도 무조건 다르다. 부동소수점에서 $0.1 + 0.2$가 0.3이 되지 않는 일이 여기서는 없다.

밑을 2로 하면 이것이 깨진다. $2^{1} = 2^{0} + 2^{0}$이므로 다른 배열이 같은 값이 된다. 밑을 e로 잡은 이유가 여기 있다.

5.3. 크기 비교

크기 비교는 값을 봐야 한다. 큰 지수 슬롯부터 차이를 쌓아 가다가, 남은 슬롯을 모두 더해도 역전할 수 없을 만큼 차이가 확대되면 중단한다. 대부분 첫 몇 슬롯에서 끝난다.

5.4. 정규화

곱셈이 이어지면 슬롯이 증가한다. 층이 끝날 때마다 값이 아주 작은 슬롯을 절단한다. 오차는 이때만, 정한 만큼만 생긴다. 밖에서 실수가 들어올 때 양자화 스텝(실수를 정해진 간격의 값으로 반올림할 때 그 간격)에 맞추는 것까지 해서 오차가 생기는 곳은 두 군데뿐이다.

5.5. 버킷 내적

가중치가 로그 4비트면 지수 슬롯이 8개뿐이다. 내적을 슬롯별로 묶으면 이렇게 된다.

$$\sum_{i} a_i \, e^{k_i} = \sum_{k} e^{k} \Big(\sum_{i:\, k_i = k} a_i \Big) \tag{6}$$

활성값을 슬롯 8개 버킷(같은 지수끼리 모아서 더하는 통)에 나눠 더하기만 하면 내적 결과가 정확한 반야수로 나온다. 길이 4096 내적이 덧셈 4096번과 슬롯 8개 정리로 끝난다. 곱셈은 한 번도 없다.

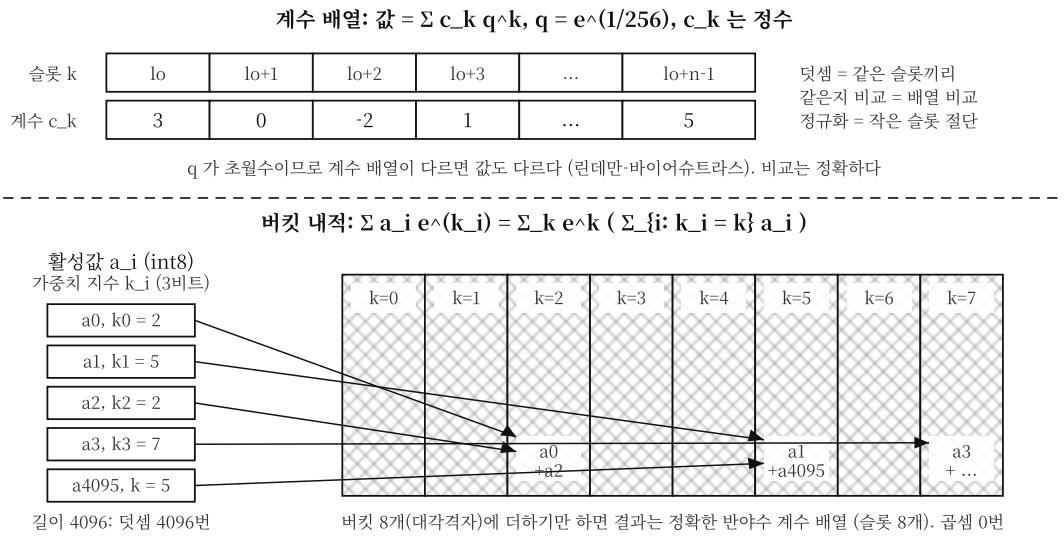


Figure 4. 계수 배열과 버킷 내적 - 값은 지수 슬롯마다 붙은 정수 계수 배열이며(위), 길이 n 내적은 활성값을 가중치 지수별 버킷 8개(대각격자)에 더하기만 하면 정확한 반야수 계수 배열로 나온다(아래). 곱셈은 한 번도 없다.

6. 혼합 구조와 정수 로그

기존 로그 수체계(LNS)는 덧셈을 로그 안에서 하려고 보정표를 사용한다. 본 논문은 덧셈을 로그 밖에서 한다. 그러면 보정표가 필요 없다.

6.1. 구조

가중치는 부호 1비트와 지수 3비트, 합쳐서 4비트다. 값은 플러스 마이너스 2의 k제곱이다. 활성값은 int8 그대로 둔다. 곱셈은 시프트 한 번이고, 누적은 int32 덧셈이다. 결과는 일반 정수 그대로 다음 층으로 넘긴다.

```
// w: 4비트 로그 가중치 (위 1비트 부호, 아래 3비트 지수)
// a: int8 활성값
int acc = 0;
for (int i = 0; i < n; ++i) {
    unsigned char w = W[i];
    int p = (int)A[i] << (w & 7);      // 곱셈 대신 시프트
    acc += (w & 8) ? -p : p;           // 부호
}
```

6.2. 변환 비용

로그에서 일반 수로 가는 것은 시프트다. 일반 수에서 로그로 가는 것은 CLZ 명령 하나다. x86과 ARM 모두 1클럭이다. 소수부가 더 필요하면 최상위 비트 아래 비트를 그대로 추출하여 사용한다(미첼 근사, 1962 [2]).

```
int lg = 31 - __builtin_clz(x);     // floor(log2 x)
```

6.3. 정수 로그의 범위

로그값을 고정소수점 정수로 두면 범위가 매우 넓다. 오차는 비율로 일정하다. 큰 수든 작은 수든 오차 비율이 같다.

Table 7. 정수 로그 형식별 범위와 스텝 간격.

형식	범위	스텝 간격
int32 Q8.8	2의 플러스 마이너스 800만 제곱	0.27%
int32 Q16.16	2의 플러스 마이너스 3만 제곱	0.001%
int64 Q32.32	2의 플러스 마이너스 20억 제곱	100억분의 1% 수준

6.4. 보정표를 사용할 경우의 크기

로그 안에서 덧셈을 하려면 보정값 $f(d)$가 필요하다. 비교용으로 크기를 기재해 둔다.

$$f(d) = \log_2\left(1 + 2^{-d}\right) \tag{7}$$

소수부 비트가 F면 테이블 엔트리(표에 저장된 값 하나하나) 수는 약 $(F + 1.5)$ 곱하기 2의 F제곱이다.

Table 8. 보정표의 크기.

형식	엔트리마다 저장	직선 보간
Q8.8	약 2,400 엔트리, 5KB	필요 없음
Q16.16	약 115만 엔트리, 4.6MB	약 4,500 엔트리, 18KB
Q32.32	불가능	약 55만 엔트리, 4.4MB

반야수는 이 표를 사용하지 않는다. 표는 실수값만 담아서 1차원 환원이기 때문이다.

6.5. float도 절반은 로그다

IEEE 754 float 비트를 정수로 읽으면 대략 $\log_2(x)$다. 그래서 정수 덧셈 한 번으로 곱셈이 근사로 나온다.

```
int ia = *(int*)&a;
int ib = *(int*)&b;
int ic = ia + ib - 0x3F800000;    // 정수 덧셈만으로 a*b 근사
float c = *(float*)&ic;
```

7. LLM 적용

LLM 추론은 처음부터 끝까지 CPU로 된다. GPU가 반드시 필요한 연산은 하나도 없다. GPU는 빨리 할 뿐이다. 무거운 곳은 긴 문맥 어텐션과 긴 프롬프트 프리필 두 군데다.

7.1. 구간별 처리

Table 9. LLM 추론 구간별 연산과 처리.

구간	연산	처리	비용
Q, K, V 생성, 출력 투영, FFN	가중치와 활성값	시프트 또는 버킷 내적	경량
누적	덧셈	int32 덧셈	경량
어텐션 Q와 K, 그 결과와 V	활성값끼리	CLZ로 로그 갔다가 시프트	문맥 길이에 비례
소프트맥스	지수	누적값을 지수 슬롯에 바로 대입한다	경량
출력 top-k	순서만	정수 비교	경량

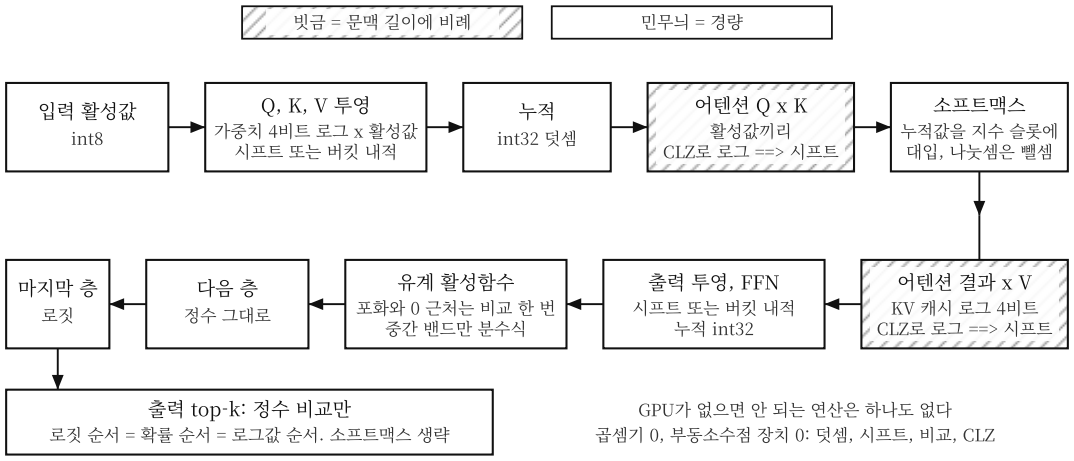


Figure 5. LLM 추론 한 층의 정수 전용 경로 - 가중치와 활성값의 곱은 시프트 또는 버킷 내적, 누적은 int32 덧셈, 활성값끼리의 어텐션 곱은 CLZ로 로그에 갔다가 시프트, 소프트맥스는 누적값을 지수 슬롯에 대입, 출력은 정수 비교로 끝난다. 문맥 길이에 비례하는 구간(빗금)은 어텐션 두 곳뿐이다.

7.2. top-k는 비교로 끝난다

e^x도 log도 단조 증가 함수다. 그래서 로짓 순서가 곧 확률 순서고 로그값 순서다. 가장 큰 것 하나만 선별하면 소프트맥스는 실행하지 않는다. 확률로 표본 추출할 때도 어휘 수만 개가 아니라 위에서 40~50개만 확률로 바꾼다.

7.3. 소프트맥스와 미분

e 지수 형태는 미분해도 형태가 변하지 않는다.

$$\frac{d}{dx}e^x = e^x, \qquad \frac{d}{dx}e^{kx} = ke^{kx}, \qquad \frac{d}{dx}e^{ix} = ie^{ix} \tag{8}$$

순방향은 $\log p_i = x_i - \log(\text{전체 합})$이라 나눗셈이 뺄셈 한 번이다. 역방향(학습) 기울기는 $p_i(\delta_{ij} - p_j)$라서 이미 구한 p만 사용한다. 새로 지수를 구할 일이 없다.

7.4. KV 캐시와 문맥 길이

KV 캐시도 로그 4비트로 저장하면 메모리가 절반 아래로 감소하고 어텐션 곱셈도 시프트가 된다. K와 V를 여러 헤드가 나눠 사용하는 GQA 모델을 사용하면 캐시가 더 감소한다. 슬라이딩 윈도우로 최근 일정 길이만 보게 하면 문맥 길이에 따라 증가하는 것을 차단한다. 1~3B 작은 모델이면 프리필도 짧다.

8. 유계 활성함수

시그모이드와 tanh도 실수로 변환하지 않고 반야수 안에서 처리한다.

8.1. 누적값을 지수 슬롯에 대입한다

혼합 구조에서 층 누적값 x는 일반 정수다. 이것을 반야수 지수 슬롯에 그대로 대입하면 e^x가 된다. 연산은 0번이다.

$$\sigma(x) = \frac{e^0}{e^0 + e^{-x}}, \qquad \tanh(x) = \frac{e^0 - e^{-2x}}{e^0 + e^{-2x}} \tag{9}$$

분자와 분모는 두 항짜리 계수 배열이다. 분수식 그대로 유지한다. $2x$는 시프트 한 번이다.

8.2. 유계는 비교 한 번으로

입력이 조금만 커져도 출력은 포화값(입력이 아무리 커져도 더 변하지 않는 끝값, 1 또는 -1)에 수렴한다. x의 정수 크기만 보면 대부분 결정된다.

Table 10. 입력 구간별 유계 함수 처리.

구간	tanh	σ	비용
x가 8보다 크다	정확히 1	정확히 1	비교 한 번
x가 0 근처	x 그대로	$1/2 + x/4$	시프트 한 번
x가 -8보다 작다	정확히 -1	정확히 0	비교 한 번
중간 밴드	분수식	분수식	반야수 두 항

Q8.8에서 x가 8을 넘으면 tanh는 Q8.8 정밀도 안에서 정확히 1이다. 근사가 아니라 참값이다. 계산이 필요한 것은 중간 밴드(포화도 아니고 0 근처도 아닌 입력 구간)뿐이다.

```
// x: 누적 정수 (Q8.8), SAT = 8 * 256
if (x >  SAT) return BANYA_ONE;                  // 포화 1
if (x < -SAT) return BANYA_NEG_ONE;              // 포화 -1
if (x > -LIN && x < LIN) return x;               // 0 근처: tanh(x) = x
return banya_frac(e0 - e(-2*x), e0 + e(-2*x));   // 중간 밴드만 식으로
```

8.3. 유계는 회전의 프로젝션이다

쌍곡함수와 원함수는 90도 회전 관계다. 구더만 함수 $\mathrm{gd}(x)$를 삽입하면 tanh는 단위원 위 각도를 실수축에 프로젝션(2차원 점을 실수축 위로 수직으로 내려 그 좌표만 남기는 것)한 값이다.

$$\tanh(x) = -i\tan(ix), \qquad \tanh(x) = \sin(\mathrm{gd}(x)), \qquad \mathrm{gd}(x) = 2\arctan(\tanh(x/2)) \tag{10}$$

tanh가 -1과 1 사이에 제한되는 것은 단위원 반지름이 1이라서다. 그래서 tanh 출력은 크기 1, 각도 $\mathrm{gd}(x)$인 반야수로 저장한다. 크기가 1이니 유계는 자료형이 자동으로 지킨다. 각도는 16비트라 오버플로되면 자동으로 순환된다. 실수 지수(쌍곡)를 허수 지수(회전)로 옮기는 것, 이것이 2차원을 그대로 유지한다는 말의 수학적 형태다.

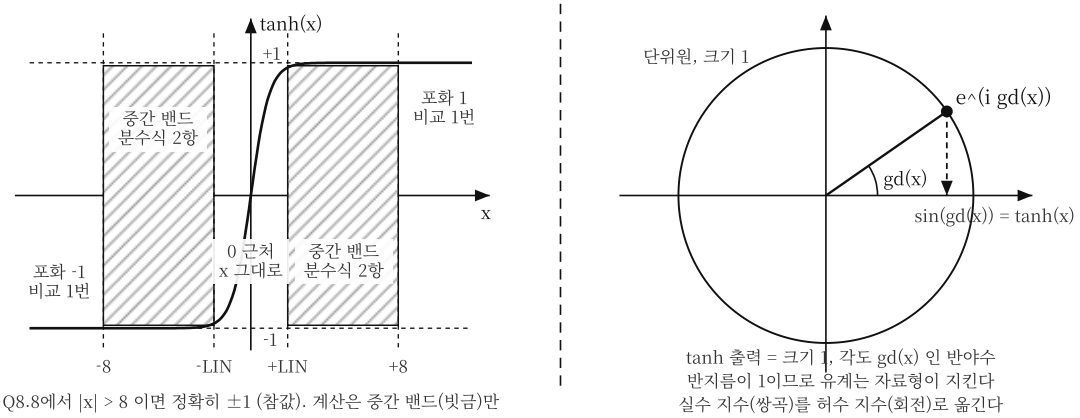


Figure 6. 유계 활성함수의 세 구간과 회전 프로젝션 - 왼쪽은 $\tanh(x)$의 포화 구간, 0 근처 선형 구간, 중간 밴드(빗금)이며 중간 밴드만 분수식 계산이 필요하다. 오른쪽은 단위원 위 각도 $\mathrm{gd}(x)$를 실수축에 프로젝션하면 $\tanh(x)$가 되는 구조이며, 크기 1이므로 유계는 자료형이 지킨다.

8.4. 활성값 분류

다음 층 가중치와 곱할 때 활성값을 세 경로로 구분한다.

1. 포화값 1, -1: 가중치를 그대로 사용한다. 곱셈 0번, 부호만 붙인다.
2. 0 근처 값: 정수 그대로 시프트 길로 간다.
3. 중간 밴드 값: 분수식 반야수로 들고 버킷 내적에 넣는다.

변환에 드는 자원은 세 번째 경로뿐이다. 그 비율은 엔진에서 활성값 분포를 측정한다.

8.5. 반야수 전용 활성함수

ReLU는 부호 비트만 보면 되므로 비용이 0이다. tanh 대신 각도 클램프를 사용할 수 있다. 16비트 각도를 플러스 마이너스 90도에서 절단한다. 구더만 계산 없이 정수 하나로 유계 활성함수가 된다.

```
unsigned short angle_clamp(short theta) {       // 90도 = 16384
    if (theta >  16384) theta =  16384;
    if (theta < -16384) theta = -16384;
    return (unsigned short)theta;
}
```

9. 선행 연구와 차이

로그로 신경망 곱셈을 없애는 연구는 있다 [5,6]. 대부분 전용 칩 쪽이다 [7,8,10]. 2차원 수를 회전으로 다루고 식 자체를 값으로 보유하는 수체계를 LLM에 사용한 경우는 없다.

Table 11. 선행 연구와 제품, 그리고 본 논문과 다른 점.

연구 또는 제품	때	내용	본 논문과 다른 점
네이피어 로그표, 계산자 [1]	1614	곱셈을 로그 덧셈으로	출발점이 같다
미첼 근사 [2]	1962	비트로 log2 근사	변환에 가져다 사용한다
클렌쇼, 올버의 레벨-인덱스 산술 [3]	1984	e를 거듭 올려 수를 표현	2차원, 회전, LLM이 없다
정확한 실수 계산		식을 값으로 보존	범용 계산용, 속도가 목표 아님
LNS-Madam [5]	2021	NVIDIA와 Caltech, LNS로 저정밀 학습	GPU급 하드웨어
PNNL LLM-LNS [7]	2024	LLM 이상치와 LNS를 맞춤	하드웨어 공동설계
QAA-LNS [8]	2025	로그 덧셈을 직선 조각으로 근사, MAC 면적 최대 32.5%, 에너지 최대 53.5% 절감	본 논문은 로그 덧셈을 하지 않는다
Tensordyne (옛 Recogni) Napier [9]	2025	로그 연산 추론 칩, NVIDIA 대비 와트당 토큰 17배 주장	전용 칩, 본 논문은 일반 CPU
llama.cpp [12]	2023	CPU 4비트 선형 양자화	선형 간격, 곱셈을 사용한다
Microsoft BitNet b1.58 [13]	2024	$-1, 0, 1$ 가중치로 곱셈 제거	값이 3개, 본 논문은 로그 16개
Microsoft T-MAC [14]	2024	저비트 곱셈을 테이블 룩업으로	선형 가중치, 1차원

다른 점은 네 가지다.

1. 수를 2차원 반야수로 두고 연산을 회전으로 한다. 기존 연구는 전부 1차원 실수 위에서 작동한다.
2. 로그 덧셈을 근사하지 않고 회피한다. 누적은 로그 밖에서, 합은 식 그대로.
3. 일반 CPU 소프트웨어다. 정수 전용 로그 LLM 엔진은 비어 있는 위치다.
4. e의 초월성으로 같은지 비교를 정확하게 하고, 추론, 학습, 분수 미분을 한 자료형에서 처리한다.

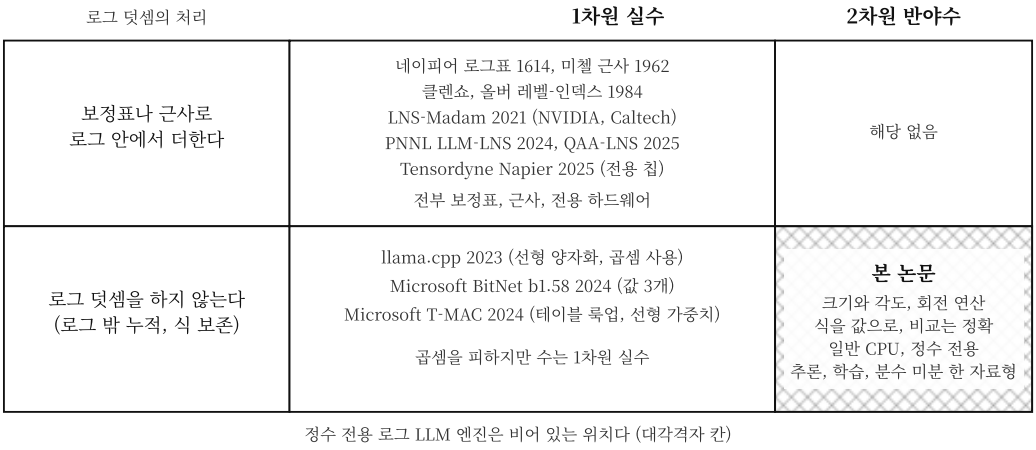


Figure 7. 선행 연구의 위치 지도 - 가로축은 수의 차원(1차원 실수, 2차원 반야수), 세로축은 로그 덧셈의 처리(보정표나 근사, 로그 밖 누적과 식 보존)이다. 기존 연구와 제품은 전부 1차원 실수 칸에 위치하고, 2차원이면서 로그 덧셈을 회피하는 칸(대각격자)은 본 논문만 위치한다.

10. 계산량 추정

연산기만 보면 에너지가 2~4배 감소하고, 비트 수까지 감소시키면 시스템 전체로 수 배에서 10배대를 목표로 한다.

10.1. 연산 한 번의 에너지 (Horowitz 2014, 45nm)

Table 12. 연산 한 번의 에너지 [4].

연산	덧셈 pJ	곱셈 pJ	곱셈이 몇 배
8비트 정수	0.03	0.2	약 7배
32비트 정수	0.1	3.1	약 30배
FP16	0.4	1.1	약 3배
FP32	0.9	3.7	약 4배

DRAM에서 32비트 하나 읽는 데 약 640pJ가 든다. FP32 곱셈의 170배다. LLM이 토큰을 하나씩 생성할 때 병목은 연산이 아니라 메모리다. 그래서 가중치 비트 수를 감소시키는 것이 가장 크다.

10.2. CPU 추론 속도 (7B 모델)

가중치를 4비트로 하면 약 3.5GB다. 일반 DDR5 듀얼 채널이 초당 약 80GB이므로 토큰 생성 속도는 초당 20개 안팎이 상한이다. 프리필은 토큰당 약 14 GFLOP이라 1,000토큰 프롬프트는 CPU에서 수 초에서 수십 초다.

10.3. 어텐션 비중 (7B, 32층, 차원 4096)

Table 13. 문맥 길이에 따른 어텐션 비중과 KV 캐시.

문맥 길이	가중치 연산 대비 어텐션	KV 캐시 8비트
4천 토큰	약 7%	약 1GB
3만 토큰	약 60%	약 8GB

10.4. 환경별 이득

Table 14. 실행 환경별 이득과 이유.

환경	이득	이유
전용 칩	연산기 2~4배, 시스템 수 배에서 10배대	곱셈기 제거와 저비트
일반 CPU	메모리와 대역폭	CPU는 곱셈도 SIMD 1클럭이다
FPU 없는 기계	실행 자체	정수만 사용한다

일반 CPU에서는 테이블 룩업(미리 계산해 둔 표에서 값을 찾아 사용하는 것)이 메모리 접근이라 느려질 수 있다. 버킷 내적과 SIMD 덧셈처럼 테이블 없는 경로가 유리하다.

11. 맺음말

11.1. 수는 2차원이다

지금까지 컴퓨터는 수를 1차원 실수로 환원하여 계산했다. 그때마다 각도를 소실하고, 미분할 때마다 상수를 소실했다. 반야수는 수를 2차원 그대로 유지한다. 곱셈은 덧셈이 되고, 미분과 적분은 회전이 되고, 합은 식 그대로 남는다. 변환도 손실도 없다.

11.2. 수의 실체는 반야식 공리 1이다

수는 반야식 공리 1과 같다 [11].

$$\delta^2 = (\text{time} + \text{space})^2 + (\text{observer} + \text{superposition})^2 \tag{11}$$

공리 1은 서로 직교하는 두 성분과 그 크기 δ로 이루어져 있다. 반야수의 크기와 각도가 바로 이 구조다. 즉 수는 값이 아니라 토폴로지다. 공리 1이 보여주는 반야식이 수의 실체이고, 반야수는 그 실체를 컴퓨터 자료형으로 옮긴 것이다.

11.3. 확장: 3차원 사원수

본 논문은 2차원 수만 다룬다. 다음 확장은 3차원이다. 3차원 회전을 다루는 사원수 형태로 식을 그대로 보유하는 것이다. 2차원 반야수가 각도 하나를 보유하듯, 사원수 반야수는 회전축과 각도를 함께 보유한다. 미분과 분수 미분은 이때 임의의 축을 기준으로 한 회전이 된다.

11.4. 다음 단계

반야수 하나는 32비트 공용체에 들어가고, AVX2 명령 하나가 반야수 곱셈 8개를 수행한다. 이것으로 GPU 없는 기계에서 정수만으로 LLM을 실행한다. 다음 단계는 이 자료형을 엔진에 이식해서 수치로 보여주는 것이다.

시리즈 2편은 이 자료형을 이식한 LLM을 실제로 실행하여 시연하고, 본 논문의 시험 항목을 전부 실측으로 검증한다. 검증 항목은 다음과 같다.

1. 반야수 곱셈, 나눗셈, 미분, 적분, 분수 미분의 역연산 복원과 SIMD 적재의 명령당 곱셈 수 (제3절, 제4절, 표 2, 표 4).
2. 계수 배열의 같은지 비교 정확성, 크기 비교의 조기 종료, 정규화 오차의 상한 (제5절, 표 6).
3. 버킷 내적의 결과가 실수 내적과 일치하는지, 길이 4096에서 곱셈 0번 (식 6).
4. 혼합 구조의 시프트 누적과 CLZ 변환 비용, 정수 로그의 범위와 스텝 간격 (제6절, 표 7).
5. top-k 순서 비교가 소프트맥스 확률 순서와 일치하는지, 소프트맥스의 뺄셈 나눗셈 (제7절).
6. 유계 활성함수의 세 구간 비율과 포화 참값, 각도 클램프 (제8절, 표 10).
7. 7B급 모델의 CPU 토큰 생성 속도, 어텐션 비중, 환경별 이득 (제10절, 표 13, 표 14).

각 항목은 본 논문의 표와 식을 그대로 시험 항목으로 사용하고, 측정값을 본 논문의 추정값 옆에 병기한다.

1. Napier, J. *Mirifici Logarithmorum Canonis Descriptio*; Andrew Hart: Edinburgh, UK, 1614.
2. Mitchell, J.N. Computer Multiplication and Division Using Binary Logarithms. *IRE Trans. Electron. Comput.* **1962**, *EC-11*, 512-517.
3. Clenshaw, C.W.; Olver, F.W.J. Beyond Floating Point. *J. ACM* **1984**, *31*, 319-328.
4. Horowitz, M. Computing's Energy Problem (and what we can do about it). In Proceedings of the IEEE International Solid-State Circuits Conference (ISSCC), San Francisco, CA, USA, 9-13 February 2014; pp. 10-14.
5. Zhao, J.; Dai, S.; Venkatesan, R.; Zimmer, B.; Ali, M.; Liu, M.-Y.; Khailany, B.; Dally, W.J.; Anandkumar, A. LNS-Madam: Low-Precision Training in Logarithmic Number System Using Multiplicative Weight Update. *IEEE Trans. Comput.* **2022**, *71*, 3179-3190.
6. Sanyal, A.; Beerel, P.A.; Chugg, K.M. Neural Network Training with Approximate Logarithmic Computations. In Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Barcelona, Spain, 4-8 May 2020; pp. 3122-3126.
7. Haghi, P.; Wu, C.; Azad, Z.; Li, Y.; Gui, A.; Hao, Y.; Li, A.; et al. Bridging the Gap Between LLMs and LNS with Dynamic Data Format and Architecture Codesign. In Proceedings of the 57th IEEE/ACM International Symposium on Microarchitecture (MICRO), Austin, TX, USA, 2-6 November 2024; pp. 1617-1631.
8. Hamad, H.; Qiu, Y.; Beerel, P.A.; Chugg, K.M. Bitwidth-Specific Logarithmic Arithmetic for Future Hardware-Accelerated Training. arXiv:2510.17058, 2025.
9. Tensordyne. Tensordyne Announces Breakthrough Inference System to End AI's Speed vs. Cost Trade-Off (Napier, CEO Marc Bolitho). Company announcement, 2025. Available online: https://www.tensordyne.ai/ (accessed on 30 October 2026).
10. Aussie AI. Zero-Multiplication Models, Logarithmic Models. Available online: https://www.aussieai.com/ (accessed on 30 October 2026).
11. Han, H. Banya Framework: An Axiom-Based Science Mining Engine for Deriving Fundamental Physical Constants. Zenodo **2026**. Available online: https://doi.org/10.5281/zenodo.19174651 (accessed on 30 October 2026). 공리 원문: https://ubmscoin.github.io/banya
12. Gerganov, G.; et al. llama.cpp: LLM inference in C/C++. GitHub repository, 2023. Available online: https://github.com/ggml-org/llama.cpp (accessed on 30 October 2026).
13. Ma, S.; Wang, H.; Ma, L.; Wang, L.; Wang, W.; Huang, S.; Dong, L.; Wang, R.; Xue, J.; Wei, F. The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits. arXiv:2402.17764, 2024.
14. Wei, J.; Cao, S.; Cao, T.; Ma, L.; Wang, L.; Zhang, Y.; Yang, M. T-MAC: CPU Renaissance via Table Lookup for Low-Bit LLM Deployment on Edge. arXiv:2407.00088, 2024.